```
import kagglehub

# Download latest version
path = kagglehub.dataset_download("hassaanmustafavi/phishing-urls-dataset")

print("Path to dataset files:", path)
```

```
Using Colab cache for faster access to the 'phishing-urls-dataset' dataset.
Path to dataset files: /kaggle/input/phishing-urls-dataset
```

```
import os

print(os.listdir(path))
```

```
['url_dataset.csv']
```

```
import pandas as pd

df = pd.read_csv(path + "/url_dataset.csv")
print(df.head())
```

```
                        url        type
0     https://www.google.com  legitimate
1    https://www.youtube.com  legitimate
2   https://www.facebook.com  legitimate
3      https://www.baidu.com  legitimate
4  https://www.wikipedia.org  legitimate
```

```
df['label'] = df['type'].map({
    'legitimate': 0,
    'phishing': 1
})
```

```
import re

def extract_features(url):
    url = url.lower()

    suspicious_words = [
        'bank', 'secure', 'account', 'login',
        'verify', 'update', 'free', 'bonus',
        'ebay', 'paypal', 'signin', 'confirm'
    ]

    return [
        len(url),                               # طول اللينك
        url.count('.'),                         # عدد النقاط
        url.count('-'),                         # عدد الشرطات
        url.count('/'),                         # depth
        len(url.split('.')),                    # subdomains

        1 if '@' in url else 0,                 # وجود @
        1 if 'https' in url else 0,             # https
        1 if 'http://' in url else 0,           # مش secure

        1 if re.search(r'\d+\.\d+\.\d+\.\d+', url) else 0,  # IP

        1 if any(word in url for word in suspicious_words) else 0,  # كلمات مشبوهة

        sum(word in url for word in suspicious_words),  # عدد الكلمات المشبوهة

        len(re.findall(r'\d', url)),            # عدد الأرقام

        1 if len(url) > 75 else 0               # لينك طويل جدًا
    ]
```

```
X = df['url'].apply(extract_features).tolist()
y = df['label']

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

model = RandomForestClassifier()
model.fit(X_train, y_train)
```

```
▾ RandomForestClassifier
RandomForestClassifier()
```

```
from sklearn.metrics import classification_report

y_pred = model.predict(X_test)
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.99      1.00      0.99     69087
           1       0.99      0.96      0.98     20949

    accuracy                           0.99     90036
   macro avg       0.99      0.98      0.99     90036
weighted avg       0.99      0.99      0.99     90036
```

```
import pickle
pickle.dump(model, open("model.pkl", "wb"))
```

```
from google.colab import files
files.download("model.pkl")
```